

Unrolled List

–Search Element

–Time Complexity Analysis

Program

```
// Function to search for an element
void searchElement(int val) {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {
                cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
                return;
            }
        }
        temp = temp->next;
        nodeCount++;
    }

    cout << "Element " << val << " not found in the list.\n";
}

... .

    case 4:
        cout << "Enter integer value to search: ";
        cin >> value;
        searchElement(value);
        break;
```

Time Complexity Analysis

- *n*: Total number of elements.
- *N*: Total number of nodes ($N \approx \left\lceil \frac{n}{4} \right\rceil$).
- *B*: Maximum node capacity (4).

1. Best Case Time Complexity: $O(1)$

The best – case scenario occurs when the algorithm performs the minimum possible number of operations.

- **Absolute Best Case (Empty List)**: *The function checks `if (head == NULL)`. If true, it prints an error message and immediately executes a `return;` statement.*

This takes strictly constant time.

- **Not Absolute Best Case (First Element Match)**: *The list is populated, and the target value happens to be **located at index 0** of the very first node (`temp → elements[0] == val`). The inner loop matches on the first iteration, prints the success message, and exits immediately via `return;`.*

Line of Code → Absolute Best Case < Not Absolute Best Case.

- **Result**: *Because the loops barely execute or do not execute at all, the time complexity is $O(1)$.*

2. Worst Case Time Complexity: $O(n)$ (or $O(N)$)

1. Node Traversal (Outer Loop)

```
while (temp != NULL) {  
    ...  
    temp = temp->next;  
    nodeCount++;  
}
```

→ Because the element is never found, the function never hits the **return;** statement inside the loop.

→ Therefore, the outer loop must visit every single node in the list from **head** until **temp** becomes **NULL**.

```
temp = temp->next;
```

- **Temp = Temp → Next**, this assignment runs, till **Temp = NULL** hence : $1 + 1 + 1 \dots N = N$ times.

```
nodeCount++;
```

- Increments the **nodeCount** till nodecounts, runs: $1 + 1 + 1 \dots N = N$ times.

→ **Cost:** Let c_n be the physical cost of hopping to the next node and incrementing the counter.

$$\text{Nodes Traversed Cost} = c_n \times N$$

$$\text{or, Exactly: } N + N = 2N, \text{ where } c_n = 2.$$

2. Element Comparisons (Inner Loop)

```
for (int i = 0; i < temp->numElements; i++) {  
    if (temp->elements[i] == val)
```

- *Inside each visited node, the inner loop executes for every active element.*
- *Because the if condition always evaluates to False, the loop never breaks early. It checks k_1 elements in Node 1, k_2 elements in Node 2, all the way up to Node N.*
 - $1 + 1 + 1 \dots n \text{ times} = 1 \times n = n.$
- *The sum of all these elements across all N nodes is exactly n.*
- **Cost:** *Let c_s be the cost of running the if comparison.*

$$\text{Elements Compared Cost} = c_s \times n$$

or, Exactly: $1 \times n = n$, where $c_s = 1$.

3. Success Execution (Inside the if block)

```
cout << "Element " << val << " found in Node " <<  
nodeCount << " at index " << i << ".\n";  
return;
```

- *Because the element is not found, the if block is entirely bypassed.*
- **Cost:** 0

$$T_{\text{worst}} = \text{Nodes Traversed Cost} + \text{Elements Compared Cost} \\ + \text{Success Execution Cost}$$

$$T_{\text{worst}} = (c_n \times N) + (c_s \times n) + 0$$

Or , exactly,

$$T_{\text{worst}} = 2N + 1n + 0$$

$$\Rightarrow T_{\text{worst}} = 2N + n$$

Applying Big – O and dropping the constants we get:

$$T_{\text{worst}} = O(N) + O(n)$$

Consolidate Proportional Variables: $N \approx \left\lceil \frac{n}{4} \right\rceil$, meaning the total number of node blocks scales directly and proportionally with the total number of elements. Because **N** and **n** grow at the exact same linear rate, **O(N)** is mathematically equivalent to **O(n)**. They absorb into each other.

Therefore, by combining the **2N** node operations with the **n** element comparisons, the final Worst Case Time Complexity definitively simplifies to **O(n)** (or **O(N)**).

Average Case Time Complexity

A. Average Element Comparisons (Inner Loop)

A. 1. Based on Loop

```
for (int i = 0; i < temp->numElements; i++) {  
    if (temp->elements[i] == val) {  
        cout << "Element " << val << " found in Node " <<  
nodeCount << " at index " << i << ".\n";  
        return;  
    }  
}
```

*If there are **n** total elements across the entire unrolled linked list,
the target element could be located at any of the **n** positions.*

- If we search for the 1st element,*

```
for (int i = 0; i < temp->numElements; i++)
```

*the inner loop comparison runs **1** time.*

```
if (temp->elements[i] == val)
```

- If we search for the 2nd element, comparison runs **2** times.*
- If we search for the **n** – th element, comparison runs **n** times.*

Note : we are doing comparisons like:

Node 1 (Elements 1 through 4)

- Target is 1st element: 1 comparison.*
- Target is 2nd element: 2 comparisons.*
- Target is 3rd element: 3 comparisons.*
- Target is 4th element: 4 comparisons.*

Node 2 (Elements 5 through 8)

- **Target is 5th element: 4 comparisons (from Node 1) + 1 comparison (in Node 2) = 5 total comparisons.**
- **Target is 6th element: 4 + 2 = 6 total comparisons.**
- **Target is 7th element: 4 + 3 = 7 total comparisons.**
- **Target is 8th element: 4 + 4 = 8 total comparisons.**

Hence, here `n` elements represents total `n` elements of `N` node blocks.

Assuming a uniform distribution, the probability of the target element

being at any specific position is exactly $\frac{1}{n}$.

If there is n number of elements, we can search the target at n positions.

- **For position 1, the probability of searching target is $\frac{1}{n}$.**
- **For position 2, the probability of searching target is $\frac{1}{n}$.**
- **For position n , the probability of searching target is $\frac{1}{n}$.**

Hence probability of searching target will always remain $\frac{1}{n}$.

To find the average time complexity, we multiply the probability of the element's location by the number of iterations required to find it:

$$\text{Average} = \left(\frac{1}{n} \times 1\right) + \left(\frac{1}{n} \times 2\right) + \dots + \left(\frac{1}{n} \times n\right)$$

$$\text{Average} = \frac{1}{n} \times \{1 + 2 + 3 + \dots + n\}$$

Using the sum of an arithmetic progression formula $\left(\sum_{i=1}^n i\right)$:

$$Average = \frac{1}{n} \times \left\{ \frac{n(n+1)}{2} \right\}$$

$$Average = \frac{n+1}{2}$$

Alternate simplest approach :

A.2. Based on Position

*If there are **n** total elements in the data structure, they occupy absolute positions from **1 to n**.*

Because we assume the target element could be anywhere, every single position has the exact same, equal chance of being the target:

- *For position 1, the probability of searching is $\frac{1}{n}$.*
- *For position 2, the probability of searching is $\frac{1}{n}$.*
- *For position n, the probability of searching is $\frac{1}{n}$.*

*Hence, the probability always remains exactly $\frac{1}{n}$ for any position from **1 to n**.*

$$Average = Probability \times (Summation of number of Position)$$

By substituting our values into the equation:

$$Average = \frac{1}{n} \times \left(\sum_{i=1}^n i \right)$$

*Using the standard mathematical formula for the sum of the first **n** natural numbers:*

$$Average = \frac{1}{n} \times \left(\frac{n(n+1)}{2} \right)$$

The **n** in the numerator and the **n** in the denominator perfectly cancel each other out:

$$\text{Average} = \frac{n + 1}{2}$$

Cost: Let **c_s** be the cost of checking one element (the **if** comparison).
The total average comparison cost is **c_s** × $\left(\frac{n + 1}{2}\right)$.

B. Average Node Traversals (Outer Loop)

The list consists of **N** total node blocks. The outer **while** (**temp** ≠ **NULL**) loop hops from node to node to locate the block containing the target element.

- If the element is in the 1st node, the loop runs 1 time.
- If the element is in the 2nd node, the loop runs 2 times.
- If the element is in the **N** – th node, the loop runs **N** times.

Because the elements are distributed evenly, the probability of the element being in any specific node is $\frac{1}{N}$.

We calculate the average using the exact same statistical logic as the elements:

$$\text{Average Traversals} = \left(\frac{1}{N} \times 1\right) + \left(\frac{1}{N} \times 2\right) + \dots + \left(\frac{1}{N} \times N\right)$$

$$\text{Average Traversals} = \frac{1}{N} \times \sum_{i=1}^N i$$

$$\text{Average Traversals} = \frac{1}{N} \times \frac{N(N + 1)}{2}$$

$$\text{Average Traversals} = \frac{N + 1}{2}$$

Cost: Let c_n be the cost of hopping from one node to the next. The total average traversal cost is $c_n \times \left(\frac{N+1}{2}\right)$.

To find the total average time (T_{avg}), we add the traversal cost and comparison cost together:

$$T_{avg} = c_n \times \left(\frac{N+1}{2}\right) + c_s \times \left(\frac{n+1}{2}\right)$$

To express this in standard Big – O notation, we strip away all constant multipliers (like $\frac{1}{2}$), hardware costs (c_n, c_s), and lower – order additive constants:

$$T_{avg} = O(N) + O(n)$$

Because the total number of nodes is $N \approx \left\lceil \frac{n}{4} \right\rceil$, N is directly proportional to n . Traversing N nodes takes the same linear time as searching n elements, making $O(N)$ mathematically equivalent to $O(n)$.

Because $O(n)$ is the term with the highest growth rate, it absorbs all constants. Therefore, the final Average Case Time Complexity for searching the list is definitively $O(n)$ (or $O(N)$).
